MÁSTER EN ANÁLISIS DE MALWARE Y REVERSING
MÓDULO 7. DEBUG Y ANÁLISIS DEL SISTEMA OPERATIVO (LINUX)
ROOTKITS Y OTROS ATAQUES

MÁSTER EN *ANÁLISIS DE MALWARE Y REVERSING*

Campus Internacional CIBERSEGURIDAD

ENIIT INNOVA IT BUSINESS SCHOOL

UCAM UNIVERSIDAD CATÓLICA DE MURCIA

INDICE DE CONTENIDOS

1. ¿Qué es un rootkit y como funciona? 3

2. ¿Existen rootkits en Linux? 3

3. Tipos de rootkits3

3.1. Rootkit de espacio usuario 4

3.2. Rootkit de espacio kernel 6

3.3. Bootkits 7

4. Detección de rootkits 7

4.1. Rootkits VS antivirus y otros mecanismos automáticos de detección 9

4.2. Herramientas automatizadas para la detección de rootkits en Linux 10

4.2.1. Herramientas de escritorio 10

4.2.2. Herramientas enfocadas a vigilancia masiva de máquinas (HIDS) 18

4.2.3. Otros sistemas similares a los HIDS 26

4.2.4. Herramientas aplicadas a contenedores 27

Bibliografía: 29

1. ¿QUÉ ES UN ROOTKIT Y COMO FUNCIONA?

Un rootkit es una "herramienta" software utilizado normalmente (si bien no siempre) con fines maliciosos para conseguir acceso privilegiado en una determinada máquina y mantenerlo de una forma continuada y por lo general, desapercibida para usuarios y administradores. Los rootkits pueden venir acompañados, aunque no en todos los casos, de código malicioso con el objetivo de facilitar/posibilitar su ejecución, ocultar su actividad, exfiltrar información, etc.

Por lo general los rootkits suelen incorporar funcionalidades extra como la creación de backdoors, keyloggers, ejecución/inyección de procesos, detección y propagación a otras máquinas accesibles, etc.

2. ¿EXISTEN ROOTKITS EN LINUX?

Aunque por cuota de mercado, tradicionalmente se ha popularizado el malware y el uso de rootkits en Windows. Sin embargo, su existencia en Linux es cada vez más popular y destaca especialmente en el caso de servidores.

Son una amenaza peligrosa para los usuarios de Linux (y otros SO) ya que lo primero que se puede hacer mediante un rootkit es cambiar la contraseña de usuario y/o administrador e inutilizar nuestro sistema.

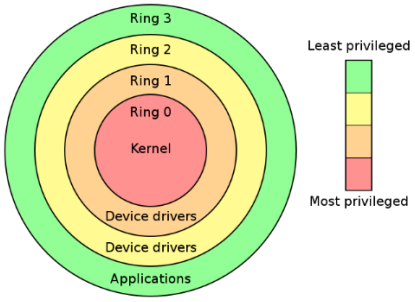
Como ejemplo, en 2014 la operación Windigo, llevada a cabo por ESET, sacó a la luz un informe sobre código malicioso que afectaba a más de 25000 servidores en el mundo, en su gran mayoría Linux. También en 2014 aparecían graves vulnerabilidades como Hearbleed o Shellshock que afectaban a servidores Unix/Linux.

3. TIPOS DE ROOTKITS

Dependiendo del tipo, existen rootkits destinados al firmware, hipervisor, el núcleo y en mayor medida, al espacio de usuario.

Como se estudió en el módulo 8.2, el sistema operativo define claramente dos áreas de memoria bien diferenciada: el espacio kernel y el espacio usuario. Típicamente estos espacios se distribuyen en "anillos de seguridad", implicando cada uno de ellos un nivel de privilegios

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA



Anillo 3	Espacio usuario. Ejecución de aplicaciones y privilegios mínimos
Anillo 2	Reservados a drivers con distintos niveles de privilegios
Anillo 1	
Anillo 0	Espacio kernel. Privilegios ilimitados

Mientras que en el espacio usuario (anillo 3), el entorno en el que se ejecutan las aplicaciones es muy controlado, ya que no acceden directamente a los recursos (memoria, disco, dispositivos, etc), sino que lo hacen a través de llamadas al sistema, en el espacio kernel (anillo 0) el nivel de privilegios es total, con lo que su impacto puede afectar a todo el sistema. La implementación en los niveles intermedios es mucho menos frecuente. Ya que esto esta íntimamente relacionado con la creación de rootkits, podemos diferenciar por tanto entre dos variantes principales de rootkits: De espacio usuario y espacio kernel.

3.1. ROOTKIT DE ESPACIO USUARIO

Se ejecuta en espacio usuario, al mismo nivel que otras aplicaciones. Habitualmente sustituye ejecutables y binarios legítimos del sistema por otros modificados, alterando su comportamiento según se desee.

Entre los principales binarios objetivo de un rootkit para conseguir su ocultación y operación se encuentran los siguientes:

- **Ficheros**: ls, df, stat, du, find, lsof, lsattr, chattr, sync…
- **Conexiones**: ip, route, neststat, lsof, nc, iptables, arp…
- **Procesos**: ps, top, pidof, kill, lsof…
- **Tareas**: crontab, at…
- **Logs**: syslogd, rsyslogd…
- **Accesos**: sshd, login, telnetd, inetd, passwd, last, lastlog,su, sudo, who, w, runlevel…

La opción de cambiar binarios es bastante tosca y fácil de detectar, por lo que frecuentemente es complementada con técnicas como la modificación al vuelo de la ejecución de estos binarios interceptando las llamadas que estos realizan a librerías dinámicas. Lógicamente esto solo funcionarán para binarios que no han sido compilados de forma estática, ya que en ese caso no realizarán llamadas a librerías externas.

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

Así, los **rootkits** utilizarán interceptar librerías cargadas dinámicamente a través de diferentes métodos, como por ejemplo la lectura de variables del sistema como **LD_PRELOAD, LD_LIBRARY_PATH**, o mediante ficheros de caché **(/etc/ld.so.cache**) y configuración **(/etc/ld.so.preload**). El objetivo desviar llamadas a las librerías legítimas hacia otras manipuladas o bien precargar directamente aquellas modificadas.

En la siguiente imagen se muestra la salida de strace para la ejecución de /bin/ls, obsérvese la comprobación de ficheros realizada para la librería ld.so. Esta (ld.so.cache) puede haber sido modificada para ejecutar acciones maliciosas.

```
root@[beholder]# strace /bin/ls
execve("/bin/ls", ["/bin/ls"], [/* 24 vars */]) = 0
brk(0)                                  = 0xd98000
mmap(NULL, 4096, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f68f4c49000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
open("/etc/ld.so.cache", O_RDONLY)      = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=30085, ...}) = 0
mmap(NULL, 30085, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f68f4c41000
close(3)                                = 0
open("/lib64/libselinux.so.1", O_RDONLY) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\320X\0\0\0\0\0\0"..., 832) = 832
fstat(3, {st_mode=S_IFREG|0755, st_size=122040, ...}) = 0
mmap(NULL, 2221912, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f68f480c000
```

A continuación, se muestra un ejemplo de interceptación de librerías mediante el uso de la variable de entorno **LD_PRELOAD**:

En este caso, el binario **/bin/ls** ha sido compilado dinámicamente, por lo que durante su ejecución se llamará a varias librerías externas:

```
root@[beholder]# ldd /bin/ls
        linux-vdso.so.1 =>  (0x00007fff3dbff000)
        libselinux.so.1 => /lib64/libselinux.so.1 (0x00007ff19038a000)
        librt.so.1 => /lib64/librt.so.1 (0x00007ff190182000)
        libcap.so.2 => /lib64/libcap.so.2 (0x00007ff18ff7d000)
        libacl.so.1 => /lib64/libacl.so.1 (0x00007ff18fd75000)
        libc.so.6 => /lib64/libc.so.6 (0x00007ff18f9e1000)
        libdl.so.2 => /lib64/libdl.so.2 (0x00007ff18f7dc000)
        /lib64/ld-linux-x86-64.so.2 (0x00007ff1905b2000)
        libpthread.so.0 => /lib64/libpthread.so.0 (0x00007ff18f5bf000)
        libattr.so.1 => /lib64/libattr.so.1 (0x00007ff18f3ba000)
root@[beholder]#
```

A su vez, estas librerías ejecutan funciones que pueden pertenecer a otras librerías compilada también dinámicamente. En el caso de **/bin/ls** una de ellas es la función **strlen**, perteneciente a la librería strings.h. La idea es interceptar alguna llamada a la librería que contiene esa función (conocido típicamente como hooking) y modificar el comportamiento de la misma para ejecutar el comportamiento deseado.

Una de las posibilidades para realizar la interceptación de una llamada a función, es el uso de la variable de entorno **LD_PRELOAD**, para ello, será necesario crear una librería propia que defina

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

la función a suplantar, en este caso **strlen** y adaptar su comportamiento al objetivo deseado. Por ejemplo, en el caso de ls, evitando que muestre la información correcta en determinados directorios:

```
1  #include <stdio.h>
2
3  int strlen(const char *c) {
4          puts("No hay ficheros en este directorio \n" );
5          exit(1);
6  }
7
```

Para terminar, compilar el código con los flag apropiados para generar una librería dinámica:

```
gcc -shared -fPIC -Wall -o mi_ls.so mi_ls.c
```

En siguiente lugar, será necesario asegurar que el ejecutable encuentre la función modificada antes que la original. Aquí es donde entra en juego la variable **LD_PRELOAD.** Para ello exportamos dicha variable con el path a la librería modificada, lo que fuerza a priorizar su uso ante el sistema. Una vez hecho esto, solo falta probar la efectividad del *hook* invocando al comando ls del sistema:

```
root@[beholder]# ls
mi_ls.c  mi_ls.so  mi_print.c  mi_print.o  mi_print.so  saludo  saludo.c
root@[beholder]# export LD_PRELOAD=$PWD/mi_ls.so
root@[beholder]# ls
No hay ficheros en este directorio

root@[beholder]#
```

Como se puede comprobar, la salida por pantalla es el resultado de ejecutar la función strlen() modificada del ejemplo.

Como se puede imaginar, este tipo de técnicas, aunque muy extendidas entre los rootkits, son sencillas de detectar mediante comprobaciones básicas de la integridad de los binarios (principalmente hashing), verificación de variables de entorno que modifiquen el path, exploración de enlaces simbólicos y rutas, análisis de configuración de librerías dinámicas, etc. De hecho, la mayoría de las soluciones de detección que se verán a continuación, se basan en gran medida en estas técnicas.

3.2. ROOTKIT DE ESPACIO KERNEL

Debido al entorno en el que se ejecutan, los rootkits de kernel serán mucho mas avanzados, potentes y difíciles de detectar que los de espacio usuario. De hecho, esto hace que cada vez sea más popular la aparición de rootkits avanzados que sacan partido del espacio kernel

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

dejando un paso atrás a los rootkits de espacio usuario.

Aunque el acceso a modo kernel puede suponer un problema difícil de superar, conseguir privilegios de superusuario e instalar un rootkit en modo kernel asegura un control total del sistema, así como mayores posibilidades de ocultación. Una vez ejecutados en modo kernel, los rookits pueden modificar no solo cualquier binario o configuración del sistema, sino directamente las propias funciones y llamadas al sistema operativo. Este proceso es muy similar al mostrado en la unidad 8.3 para "hookear" llamadas al sistema durante el "kernel tainting"

El desarrollo de estos tipos de rootkits es mucho más elaborado, ya que entre otros es necesario un funcionamiento mucho más robusto a la hora de interaccionar con las llamadas y funciones del sistema para evitar errores que puedan provocar un fallo en el kernel que pudiera detener su ejecución.

3.3. BOOTKITS

Aparte de los dos tipos principales anteriores, podemos hablar de una tercera variante que va un poco mas allá con respecto a los rootkits de espacio kernel. Se trata de los rootkits con capacidad de ejecución en el arranque, también conocidos como **bootkits**. Por el entorno en el que se ejecutan, estos rootkits pueden afectar al firmware de los sistemas y sectores de arranque de sus discos, por lo que además de una dificultad de detección mucho mayor, añaden altas capacidades de persistencia y dificultad de eliminación.

Uno de los ejemplos mas significativos es Thunderstrike, un bootkit para MacBooks presentado en 2014 por Trammell Hudson que modificaba el firmware de la EFI de arranque haciéndolo resistente a la reinstalación del SO e incluso a la sustitución del disco duro.

4. DETECCIÓN DE ROOTKITS

La detección de rootkits puede ser compleja, ya que algunos son capaces incluso de identificar y corromper aquellos sistemas destinados a detectarlos. Habitualmente estos sistemas se basan en el uso de sistemas operativos alternativos y confiables, métodos de análisis basado en conductas y comportamientos inusuales, firmas y expresiones regulares y análisis de memoria y trazas.

A la hora de detectar rootkits, será necesario prestar atención al mínimo detalle que pueda ser un indicio de compromiso. Por lo general, aunque dependerá de lo minucioso del ataque, la

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

ejecución de un rootkit, dejará huellas típicas de ejecución como: procesos con usuarios y comandos poco frecuentes, logs de actividad (tanto del sistema como de aplicaciones), conexiones abiertas, etc… Todas las técnicas de depuración y obtención de información del sistema explicada en los módulos anteriores serán de especial utilidad en estos casos.

Ejemplo práctico:

Tenemos indicios de que la máquina DC-1 (Debian 7) ha podido ser atacada. Utilizar todos los métodos aprendidos en el módulo 8.3 para recolectar información que pueda sernos de utilidad para comprobarlo.

Una de las principales limitaciones con las que se encontrará cualquier analista o sistema de detección de rootkits es que se trata de aplicaciones que modifican a conciencia muchas de las herramientas y librerías de las que depende el sistema, en ocasiones llegando a modificar incluso el propio kernel (mediante módulos y métodos similares).

Podemos concluir, por tanto, que un sistema en el que se ha ejecutado un rootkit, es un entorno no confiable, por lo que **podría NO comportarse como debería**.

Por ejemplo: El rootkit podría haber alterado la salida de la herramienta **top** para ocultar el rastro de su actividad en la lista de procesos o anulado el uso de logs para algunas herramientas críticas del sistema. Así, el analista deberá considerar la posibilidad de utilizar siempre que sea posible un entorno confiable alternativo para el análisis y depuración de la máquina comprometida. En caso contrario, es recomendable intentar repetir las pruebas con tantas técnicas distintas como sea posible, ya que esto nos dará la posibilidad de detectar eventos que no hayan sido tenidos en cuenta por los sistemas de ocultación deliberada del rootkit.

Tras haber analizado el sistema en *runtime*, un buen método complementario para analizar la presencia de un rootkit en la máquina consiste en realizar un salvado de los datos, apagar la máquina y arrancar desde un medio alternativo, como por ejemplo una imagen Live de otro SO. De esta forma, el posible rootkit, no podrá ejecutarse y falsear las salidas del sistema durante el proceso de análisis. Aunque se perderán ciertos datos de tiempo real, será posible acceder a ciertos recursos que puedan incluir trazas de la ejecución del rootkit con la certeza de que no podrán ser alterados por el mismo. Medidas como por ejemplo el volcado de memoria ram o swap serán especialmente útiles además para extraer información específica de la lógica de aplicación implementada por el rootkit, siendo incluso posible a veces recuperar parte de su código.

4.1. ROOTKITS VS ANTIVIRUS Y OTROS MECANISMOS AUTOMÁTICOS DE DETECCIÓN

Los sistemas antivirus y detectores de rootkits pueden llegar a identificar fácilmente a aquellos basados en la modificación de llamadas al sistema y peticiones de bajo nivel, ya que son fáciles de firmar y comprobar así su integridad en el caso de haber sido modificadas. Sin embargo, los rootkits intentan "protegerse" así mismos monitorizando los procesos activos y pausando su actividad, si detectan un posible escaneo del sistema, hasta que este termine.

En este juego del gato y el ratón, los fabricantes de sistemas de seguridad han ido a su vez integrando mecanismos de "contrainteligencia" para la detección de este tipo de técnicas. Así, por ejemplo, si durante un escaneo un proceso detiene su actividad o realiza comportamientos sospechosos como respuesta al escaneo, será marcado automáticamente como malicioso.
Estas técnicas de detección a su vez han forzado a los atacantes a implementar mecanismos de contraataque (también llamados retro-rutinas) en los rootkits, con el objetivo de eliminar o neutralizar directamente la presencia de los sistemas de seguridad, pasando de una estrategia de ocultación a una mucho más ofensiva. Algunos de estos contraataques consisten por ejemplo en eliminar los procesos creados por el software de seguridad, eliminando por ejemplo a los antivirus de la memoria, imposibilitando su ejecución o forzando su detención.

La batalla entre los fabricantes de seguridad y creadores rootkits evoluciona constantemente con la incorporación continua de técnicas y herramientas cada vez más avanzadas por ambos bandos.

4.2. HERRAMIENTAS AUTOMATIZADAS PARA LA DETECCIÓN DE ROOTKITS EN LINUX

4.2.1. Herramientas de escritorio

ClamAV

Es un software antivirus opensource (licencia GPL) fundado en el año 2001 por Tomasz Kojm. Actualmente tiene una implantación superior a los 500 000 servidores en todo el mundo. A pesar de soportar Windows, es popularmente conocido como el "antivirus para Linux", ya que, entre otros, soporta la mayoría de distribuciones Linux o similares. Algunos ejemplos:

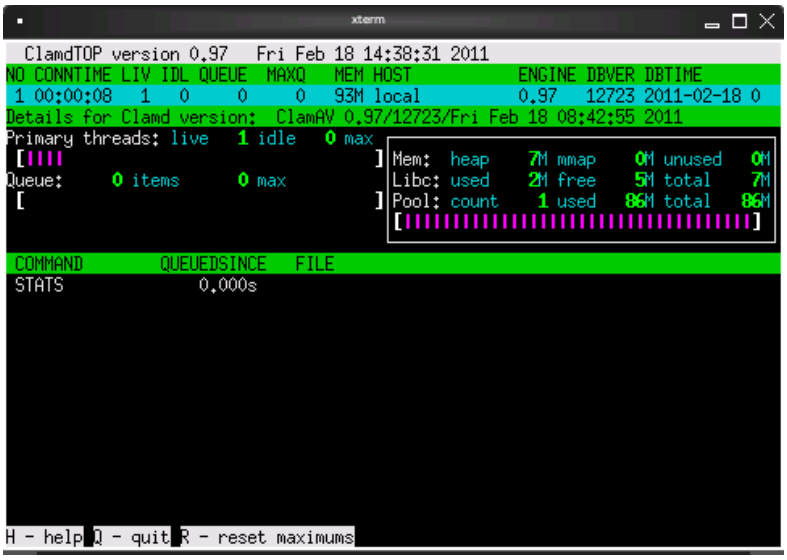
- GNU/Linux
- Solaris
- FreeBSD
- OpenBSD
- Mac OS X

Aunque originalmente fue diseñado para combatir el correo electrónico malicioso (como consecuencia de ello es ampliamente extendido entre servidores de correo), actualmente es capaz de detectar prácticamente cualquier tipo de amenaza.

Gracias a la colaboración de importantes empresas del sector, universidades y otras organizaciones, el proyecto ClamAV posee una extensa red de distribución rápida y fiable basada en mirrors, además de ampliar constantemente su base de datos amenazas, siendo capaz de detectar alrededor de 850 000 virus, gusanos y troyanos, incluyendo virus programados como macros de Microsoft Office.

Actualmente ClamAV incluye a su vez una serie de herramientas antivirus que pueden ser ejecutadas en paralelo de forma eficiente gracias a un proceso multihilo gestionado desde un monitor integrado con la línea de comentos, así como herramientas para actualizar automáticamente su base de datos de amenazas.

Una de estas herramientas es Clamtop, que permite monitorizar la actividad de uno o varios Clamd (daemons) corriendo en GNU/Linux.



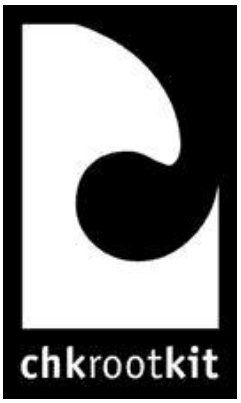
Algunas de las principales características técnicas de ClamAV:

- POSIX compatible y portable.
- Escaneo rápido multihilo.
- Escaneo de archivos y ficheros comprimidos:
    - ZIP
    - RAR
    - ARJ
    - TAR
    - Gzip
    - Bzip2
    - MS OLE2
    - MS Cabinet File
    - MS CHM
    - MS SZDD
    - BinHex
    - SIS
    - AutoIt
- Soporta plataformas de 32/64 bit.
- Soporta la mayoría de formatos de correo electrónico.
- Soporta formatos especiales como:
    - HTML
    - RTF

- PDF
- CryptFF
- SCREnc
- uuencode
- TNEF

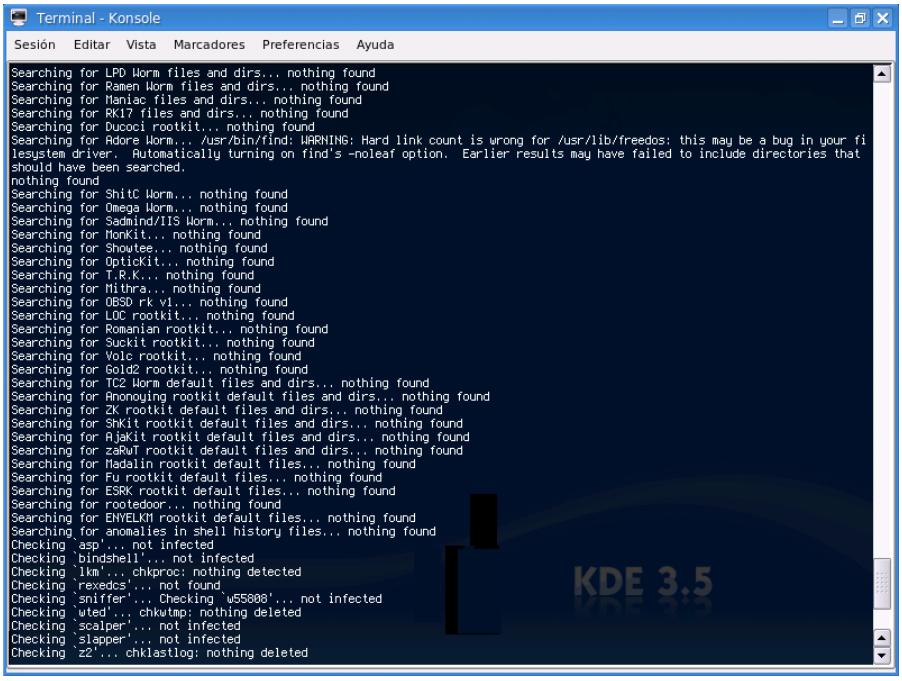
Ejemplo de instalación y uso: https://geekytheory.com/instalacion-y-uso-de-clamav-el-antivirus-de-linux

ChrootKit:



Sitio web oficial: http://www.chkrootkit.org
Enlace al código fuente: ftp://ftp.pangeia.com.br/pub/seg/pac/

Es una herramienta de consola de comandos (aunque más bien podría definirse como un shell script) de código libre bajo licencia propia desarrollado por Pangeia Informatica en 1992, que permite la detección de rootkits conocidos en sistemas operativos UNIX y derivados. Se basa en la ejecución de una batería de pruebas para detectar binarios del sistema modificados por el rootkit. Entre las herramientas utilizadas por ChkrootKit, se encuentran la mayoría de herramientas propias de los sistemas UNIX/Linux que se han explicado en las unidades anteriores, como por ejemplo **strings** o **grep** para buscar en los binarios del sistema las firmas de rootkits conocidos además de las salida de **ps** para compararlo de forma transversal con el contenido del directorio **/proc** en busca de discrepancias.



Ejemplo de ejecución en KDE 3.5

ChkrootKit funciona en los sistemas operativos **BSD, FreeBSD, OpenBSD, Linux, y Solaris** y actualmente detecta la ejecución de los siguientes rootkits:

01. lrk3, lrk4, lrk5, lrk6 (and variants);
02. Solaris rootkit;
03. FreeBSD rootkit;
04. t0rn (and variants);
05. Ambient's Rootkit (ARK);
06. Ramen Worm;
07. rh[67]-shaper;
08. RSHA;
09. Romanian rootkit;
10. RK17;
11. Lion Worm;
12. Adore Worm;
13. LPD Worm;
14. kenny-rk;
15. Adore LKM;
16. ShitC Worm;
17. Omega Worm;
18. Wormkit Worm;
19. Maniac-RK;
20. dsc-rootkit;
21. Ducoci rootkit;
22. x.c Worm;
23. RST.b trojan;
24. duarawkz;
25. knark LKM;
26. Monkit;
27. Hidrootkit;
28. Bobkit;
29. Pizdakit;
30. t0rn v8.0;
31. Showtee;
32. Optickit;
33. T.R.K;
34. MithRa's Rootkit;
35. George;
36. SucKIT;
37. Scalper;
38. Slapper A, B, C and D;
39. OpenBSD rk v1;
40. Illogic rootkit;
41. SK rootkit.
42. sebek LKM;

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

43. Romanian rootkit;
44. LOC rootkit;
45. shv4 rootkit;
46. Aquatica rootkit;
47. ZK rootkit;
48. 55808.A Worm;
49. TC2 Worm;
50. Volc rootkit;
51. Gold2 rootkit;
52. Anonoying rootkit;
53. Shkit rootkit;
54. AjaKit rootkit;
55. zaRwT rootkit;
56. Madalin rootkit;
57. Fu rootkit;
58. Kenga3 rootkit;
59. ESRK rootkit;
60. rootedoor rootkit;
61. Enye LKM;
62. Lupper.Worm;
63. shv5;
64. OSX.RSPlug.A;
65. Linux Rootkit 64Bit;
66. Operation Windigo;
67. Mumblehard backdoor/botnet;
68. Linux.Xor.DDoS Malware;
69. Backdoors.linux.Mokes.a;
70. Linux.Proxy.10
71. Rocke Monero Miner

ChkrootKit se compone de los siguientes modulos:

- **chkrootkit**: Un script en shell que verifica los binarios del sistema modificados por un rootkit.
- **ifpromisc.c**: Verifica si la interfaz de red está en modo promiscuo.
- **chklastlog.c**: Verifica por eliminaciones realizadas en el archivo lastlog.
- **chkwtmp.c**: Verifica por eliminaciones realizadas en el archivo wtmp.
- **check_wtmpx.c**: Verifica por eliminaciones en wtmpx (Solo sistemas Solaris).
- **chkproc.c**: Verifica por señales de troyanos LKM.
- **chkdirs.c**: Verifica por señales de troyanos LKM.
- **strings.c**: Reemplazo rápido y sucio de cadenas.
- **chkutmp.c**: Verifica por eliminaciones en el archivo utmp.

Una de las principales ventajas que aporta ChkrootKit de cara a conseguir un entorno confiable sobre el que ejecutar el análisis de una maquina potencialemente comprometida, es la posibilidad de ejecutarlo directamente desde una imagen Live, un disco de recuperación o incluso desde un directorio alternativo, dificultando su detección por parte de los rootkits mas avanzados y ampliando la confiabilidad de los comandos del sistema que ejecuta para obtener sus resultados.

Ejemplos de instalación y uso de ChkrootKit:

- Chkrootkit puede instalarse directamente a través de la mayoría de gestores de paquetes como por ejemplo APT:

  sudo apt install chkrootkit

- Script de instalación y configuración automática de automatizaciones como la creación de crontabs para la ejecución periódica de análisis o el envío de emails de notificación en el caso de detección de amenazas: https://web.archive.org/web/20160313220939/https://solidshellsecurity.com/tools/chkrootkit-automatic-script-installer.php

- https://linux.lat/como-detectar-rootkits-con-chkrootkit-y-rkhunter/

Rkhunter (o Rootkit Hunter)

Sitio web: http://rkhunter.sourceforge.net/
Código Fuente: https://sourceforge.net/projects/rkhunter/

Es una herramienta Unix de código libre con licencia GPL diseñada para detectar rootkits, backdoors y exploits locales. Su funcionamiento se basa en la comparación de los hashes SHA-1 (versiones anteriores usaban MD5) calculados al vuelo de los principales ficheros de la máquina sobre la que se ejecuta contra una amplia lista de bases de datos de confianza que contienen los hashes esperados para dichos ficheros. De esta forma, es posible detectar de una forma eficaz y eficiente si los ficheros críticos del sistema han sido alterados indebidamente.

Además de los hashes, rkhunter examina los directorios por efecto en los que suelen albergarse o trabajar los rootkits buscando asignaciones anómalas de permisos (por ejemplo, los permisos especiales SUID y SGID, existencia de ficheros ocultos, cadenas sospechosas en módulos del kernel y realiza tests específicos en el caso de Linux y FreeBSD.

```
Terminal — bash — bash (ttyp1) — 96x50 — ⌘1
Your system contains some unknown version numbers. Please run Rootkit Hunter
with the --update parameter or fill in the contact form (www.rootkit.nl)

Security advisories
* Check: Groups and Accounts
   Searching for /etc/passwd...                          [ Found ]
   Checking users with UID '0' (root)...                 [ OK ]

* Check: SSH
   Searching for sshd_config...
   Found /etc/sshd_config
   Checking for allowed root login... Watch out Root login possible. Possible risk!
    info:
    Hint: See logfile for more information about this issue
   Checking for allowed protocols...                     [ Warning (SSH v1 allowed) ]

* Check: Events and Logging
   Search for syslog configuration...                    [ OK ]
   Checking for running syslog slave...                  [ OK ]
   Checking for logging to remote system...              [ OK (remote logging) ]
    info: install.*                                       @127.0.0.1:32376

[Press <ENTER> to continue]


------------------------- Scan results -------------------------

MD5
MD5 compared: 0
Incorrect MD5 checksums: 0

File scan
Scanned files: 342
Possible infected files: 0

Application scan
Vulnerable applications: 0

Scanning took 129 seconds

-------------------------------------------------------------------

Do you have some problems, undetected rootkits, false positives, ideas
or suggestions?
Please e-mail me by filling in the contact form (@http://www.rootkit.nl)

-------------------------------------------------------------------
```

Ejemplo de ejecución de rkhunter en MacOS

En los últimos años rkhunter ha ganado especial importancia debido a su inclusión por defecto en algunos sistemas operativos como Fedora o Debian.

Debido a que en esencia se trata de un shell script basado en UNIX (escrito en Bourne shell y Perl), puede ser ejecutado en la mayoría de sistemas operativos base UNIX.

Lynis

Sitio web: https://cisofy.com/lynis/
Enlace al código fuente: https://github.com/CISOfy/lynis

Es una herramienta de seguridad diseñada para auditoría de sistemas operativos Linux, FreeBSD, macOS, OpenBSD, Solaris, así como gran parte de sistemas derivados de UNIX. Su objetivo final es asistir a los administradores de sistemas y profesionales de la seguridad (principalmente auditores y pentesters) en la evaluación de los sistemas y sus mecanismos de seguridad orientada al hardening de los mismos.

Lynis es de código libre con licencia GNU GPLv3 creada por Michael Boelen (también creador de rkhunter) que puede ser extendida mediante plugins extra, incluyendo además una parte *Enterprise* que incluye módulos de pago diseñados por los propios desarrolladores del core. La versión *Enterprise* incluye además un collector que permite auditar simultáneamente varias máquinas y visualizar los datos agregados a través de un servidor.

```
[+] Users, Groups and Authentication
------------------------------------
  - Search administrator accounts...                          [ OK ]
  - Checking UIDs...                                          [ OK ]
  - Checking chkgrp tool...                                   [ FOUND ]
  - Consistency check /etc/group file...                      [ OK ]
  - Test group files (grpck)...                               [ OK ]
  - Checking login shells...                                  [ WARNING ]
  - Checking non unique group ID's...                         [ OK ]
  - Checking non unique group names...                        [ OK ]
  - Checking LDAP authentication support                      [ NOT ENABLED ]
  - Check /etc/sudoers file                                   [ NOT FOUND ]

[ Press [ENTER] to continue, or [CTRL]+C to stop ]


[+] Shells
------------------------------------
  - Checking console TTYs...                                  [ WARNING ]
  - Checking shells from /etc/shells...
    Result: found 6 shells (valid shells: 6).

[ Press [ENTER] to continue, or [CTRL]+C to stop ]


[+] File systems
------------------------------------
  - [FreeBSD] Querying UFS mount points (fstab)...            [ OK ]
  - Query swap partitions (fstab)...                          [ OK ]
  - Testing swap partitions...                                [ OK ]
  - Checking for old files in /tmp...                         [ WARNING ]
  - Checking /tmp sticky bit...                               [ OK ]
```

Ejemplo de ejecución de Lynis

Con respecto a las anteriores, Lynis se trata de una herramienta de muy alto nivel, ya que realiza una evaluación mucho más amplia del sistema operativo, incluyendo por ejemplo entre sus pruebas precisamente la detección de algunas de las herramientas comentadas anteriormente como ClamAV o rkhunter. Entre los elementos del sistema que evalúa se encuentran los parámetros del kernel, mecanismos de autenticación acivos, paquetes y servicios instalados, configuración de red, syslog, mecanismos criptográficos (por ejemplo, certificados SSL/TLS instalados), etc.

La herramienta puede ser especialmente útil a la hora de automatizar procesos de auditoría, parcheo de sistemas y hardening, incluyendo algunas guías de referencia en unción de los resultados obtenidos. En función del auditor, la herramienta ejecutará distintos checks necesarios para comprobar el cumplimiento de normativas como ISO 27001, PCI-DSS 3.2 o HIPAA.

Documentación de uso oficial: https://cisofy.com/documentation/lynis/

- **Otros sistemas de detección de rootkits y amenazas en Linux**

  - **Samhain**: Es un comprobador de integridad y detector de intrusión para sistemas basados en UNIX. Pude ser utilizado tanto en modo local como multi-nodo. Mas información: https://www.la-samhna.de/samhain/

  - **Snallygaster**: Herramienta especializada en la detección de data and information leaks y descubrimiento y monitorización de información sensible en los sistemas vigilados. Mas información: https://linuxsecurity.expert/tools/snallygaster/

4.2.2. Herramientas enfocadas a vigilancia masiva de máquinas (HIDS)

Los Host-Based Intrusion Detection System (HIDS), buscan detectar anomalías que indican un riesgo potencial mediante la revisión de la actividad de la máquina (host). La diferencia con los sistemas locales es que los HIDS están diseñados para analizar simultáneamente la actividad de múltiples instancias a través de agentes locales instalados en estas, coordinando las acciones y correlando toda la información en un servidor centralizado que la analizará en tiempo real detectando riesgos y amenazas.

Las funciones de estos sistemas son muy similares a los IDS, sin embargo su enfoque es bastante diferencial, ya mientras los IDS, firewalls, Web Aplication Firewalls, etc persiguen un análisis global, enfocado generalmente en el análisis global de la red a través del análisis de puntos estratégicos de la misma (visión desde arriba), los HIDS persiguen una visión mas granular llegando al host para extraer la información en origen y agrupar después.

Si bien es cierto que la operativa y administración de estos sistemas puede ser más compleja, además de menos escalables, los resultados obtenidos pueden ser mejores que el del resto de sistemas.

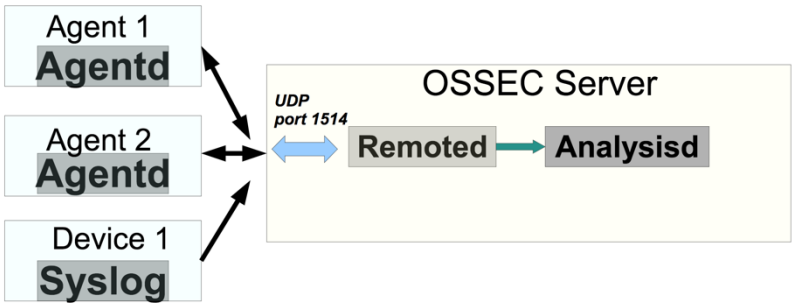
OSSEC

Sitio web: https://www.ossec.net/
Acceso al código fuente: https://github.com/ossec

OSSEC (Open Source HIDS SECurity) es un HIDS de código libre con licencia GNU GPL v2, capaz de ejecutar tareas de análisis de logs, comprobaciones de integridad de ficheros críticos, deteccioon de rootkits, generación de alertas en tiempo real e incluso ejecución de respuestas activas ante amenazas. Se encuentra disponible para una importante cantidad de SO Linux/UNIX así como para Windows.

En 2008 el proyecto OSSEC así como sus derechos fueron adquiridos por Third Brigade, Inc. quien prometió contribuir al desarrollo en la comunidad open source del proyecto así como extender el soporte comercial y formación hacia la comunidad OSSEC. Tras pasar por manos de Trend Micro (2008 a 2018) el proyecto fue entregado finalmente a la OSSEC Foundation, siendo actualmente mantenido por Atoicorp, quien además de ofrecer la versión open source y gratuita, ofrece además su propia versión comercial.

Para poder trabajar con OSSEC, es necesario conocer los conceptos de manager y agentes.



Manager: Se encarga de gestionar de forma centralizada la recepción de las alertas generadas por cada uno de los agentes. El manager es instalado en un servidor central junto con el sistema de persistencia, correlación, motor de reglas, y demás elementos.
Agente: Se encarga de recoger todas las alertas (activas) que se producen en la máquina donde se instala y enviarlas al manager.

Enlaces interesantes…

https://ossec.github.io/docs/manual/ossec-architecture.html
http://ossec.net/ossec-docs/auscert-2007-dcid.pdf

WAZUH

Sitio web: http://wazuh.com/
Enlace al Código Fuente: https://github.com/wazuh

Es una de las soluciones HIDS mas populares de código libre (licencia GNU GPL v2). Se trata de un proyecto creado en 2015 como una bifurcación de OSSEC, por la que comparte gran parte de su código básico, así como las principales funcionalidades y formas de trabajo. Cuenta con una comunidad de desarrolladores de más de 10k usuarios y es utilizado oficialmente por grandes empresas.

Los componentes básicos que componen Wazuh son: agente, servidor y el stack ELK.

Sus capacidades principales son las siguientes:

Análisis de seguridad

Permite recopilar, agregar, indexar y analizar datos de seguridad, ayudando a las organizaciones a detectar intrusiones, amenazas y anomalías de comportamiento.

Detección de intrusiones

Los agentes de Wazuh escanean los sistemas monitorizados buscando malware, rootkits y anomalías sospechosas. Pueden detectar archivos ocultos, procesos encubiertos o escuchas de red no registradas, así como inconsistencias en las respuestas a las llamadas del sistema.

Análisis de datos de registro

Los agentes de Wazuh leen los registros del sistema operativo y de las aplicaciones, y los envían de forma segura a un gestor central para el análisis y el almacenamiento basado en reglas.

Supervisión de la integridad de los archivos

Wazuh monitoriza el sistema de archivos, identificando los cambios en el contenido, los permisos, la propiedad y los atributos de los archivos a los que hay que prestar atención.

Además, identifica de forma nativa a los usuarios y las aplicaciones utilizadas para crear o modificar archivos.

Detección de la vulnerabilidad

Los agentes de Wazuh extraen los datos de inventario del software y envían esta información al servidor, donde se establece una correlación con las bases de datos CVE (Common Vulnerabilities and Exposure) que se actualizan continuamente, con el fin de identificar el software vulnerable conocido.

Evaluación de la configuración

Monitoriza los ajustes de configuración del sistema y de la aplicación para asegurarse de que cumplen con sus políticas de seguridad, estándares y/o guías de refuerzo. Los agentes realizan análisis periódicos para detectar las aplicaciones que se sabe que son vulnerables, no están parcheadas o están configuradas de forma insegura.

Respuesta a incidentes

Wazuh proporciona respuestas activas listas para ser utilizadas con el fin de llevar a cabo diversas medidas para hacer frente a las amenazas activas, como el bloqueo del acceso a un sistema desde la fuente de la amenaza cuando se cumplen ciertos criterios.

Cumplimiento de la normativa

Proporciona algunos de los controles de seguridad necesarios para cumplir con las normas y reglamentos de la industria. Estas características, combinadas con su escalabilidad y su apoyo multiplataforma, ayudan a las organizaciones a cumplir los requisitos de conformidad técnica.

Seguridad en la nube

Ayuda a monitorear la infraestructura de la nube a nivel de la API, utilizando módulos de integración que son capaces de extraer datos de seguridad de proveedores de computación en la nube conocidos, como Amazon AWS, Azure o Google Cloud. Además, Wazuh proporciona reglas para evaluar la configuración de su entorno de cloud computing, detectando fácilmente los puntos débiles.

Seguridad de los contenedores

Proporciona visibilidad de seguridad en sus hosts y contenedores Docker, monitorizando su comportamiento y detectando amenazas, vulnerabilidades y anomalías. El agente Wazuh tiene una integración nativa con el motor Docker que

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

permite a los usuarios monitorear imágenes, volúmenes, configuraciones de red y contenedores en ejecución.

Instalación de Wazuh

Aunque Wazuh puede ser instalado y configurado de forma manual, incluyendo todos los servicios que necesita (ELK, Wazuh API, Wazhu Agents, etc.), existen otras posibilidades que, aunque son menos configurables, son muy mas amigables para el usuario como por ejemplo la máquina virtual de Wazuh (https://documentation.wazuh.com/4.0/virtual-machine/virtual-machine.html) o la implementación en docker-compose (https://github.com/wazuh/wazuh-docker).

En el apéndice 1 de esta unidad se puede encontrar información detallada sobre cómo realizar una implementación completa de toda la infraestructura de Wazuh necesaria para trabajar.

Trabajar con Wazuh

Para trabajar con Wazuh es necesario conocer algunos términos.

- **Alerta**: Evento ocurrido en el sistema detectado por un agente y que debe ser enviado al manager para que este lo gestione.
- **Regla**: Una regla describe una situación en el sistema para la cual se debe generar una alerta. Normalmente una regla cuenta con un subconjunto de los siguientes campos:
    - **Id**: Identificador único de la regla.
    - **Level**: Nivel de criticidad de la regla.
    - **Frequency**: Número de veces que la regla se debe cumplir antes de generar una alerta.
    - **Timeframe**: Periodo de tiempo en el cual el atributo anterior se va a tener en cuenta. Ej: si un usuario falla 3 veces al hacer login en 5 minutos, generar una alerta. Habría que establecer Frequency a 3 y Timeframe a 5 mins.
    - **Ignore**: Periodo de espera hasta volver a analizar la regla una vez ha generado una alerta. Se usa para evitar que la misma alerta sea enviada constantemente durante un suceso de larga duración.
    - **Overwrite**: Se usa para sobre-escribir reglas definidas por OSSEC. Si queremos cambiar el comportamiento, simplemente hay que copiarla, modificarla e incluir overwrite con el valor yes.

Buena parte de la efectividad de OSSEC/Wazuh radica en una **correcta definición de sus reglas**. Definir reglas con su justo nivel de notificación es clave para no hacer inmanejable la gestión de las mismas.

1. **Donde encontramos las reglas en OSSEC…?**
   - **/var/ossec/ruleset/rules**: dentro de esa carpeta encontramos los ficheros que contienen las reglas
     que OSSEC nos ofrece. Estos ficheros se actualizan
     con la instalación de Wazuh automáticamente.
   - **/etc/rules**: en esta carpeta el usuario puede definir sus propios ficheros conteniendo sus reglas.

```
/var/ossec/
    ├ etc/
    │   ├ decoders/
    │   │       └ local_decoder.xml
    │   └ rules/
    │           └ local_rules.xml
    └ ruleset/
            ├ decoders/
            └ rules/
```

2. **Que son los decoders y donde podemos encontrarlos…?**
   Ayudan a las reglas a procesar los textos, logs, ficheros, etc. sobre los cuales se basan para generar alertas. Los decoders pueden ser encontrados en los siguientes paths:
   - **/var/ossec/ruleset/decoders**: dentro de esta carpeta encontramos los decoders propios de OSSEC. Estos ficheros se actualizan automáticamente.
   - **/etc/decoders**: aquí se pueden encontrar los decoders definidos por el usuario.
   -

Ejemplo de decoder

```
<decoder name="ossec-exampled-test-connection">
  <parent>ossec-exampled</parent>
  <prematch offset="after_parent">^test connection </prematch> <!-- offset="after_parent" makes OSSEC ignore anything matched by the parent decoder and before -->
  <regex offset="after_prematch">^from (\S+) via (\S+)$</regex> <!-- offset="after_prematch" makes OSSEC ignore anything matched by the prematch and earlier-->
  <order>srcip, protocol</order>
</decoder>
```

Descripción del decoder:

- Decoder name: nombre del decoder
- Parent: decodifica solo si el valor indicado machea con el programa que produjo el log
- Prematch: decodifica solo si el log contiene la regex de prematch
- Regex: expresión regular que debe machear con el contenido a continuación de prematch. Cada sección contenida entre paréntesis es identificada en un grupo que luego en la etiqueta order pueden ser nombrados.
- Order: reconoce cada sección entre paréntesis en regex y le asigna un nombre de izquierda a derecha. **Estos nombres luego son decodificados automáticamente por el decoder** y pueden ser utilizados, juntos con sus valores por las reglas.
  Ej: si en un log aparece el mensaje… "test conection from 127.0.0.1 via FTP" cuyo programa padre es ossec_exampled, el decoder decodificará ese mensaje y detectará automáticamente los atributos srcip=127.0.0.1 y protocol=FTP, permitiendo a las reglas trabajar con esos valores al analizar el log.

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

Ejemplo de regla

```
<rule id="5710" level="5">
  <if_sid>5700</if_sid>
  <match>illegal user|invalid user</match>
  <description>sshd: Attempt to login using a non-existent user</description>
  <group>invalid_login,authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,pci_dss_10.6.1,</group>
</rule>
```

Descripción de la regla:

- Rule Id: identificador único de la regla.
- Rule Level: nivel de criticidad que será añadido a la alerta generada.
- If_sid: machea si el valor ha macheado con la regla cuyo identificador es 5700.
- Match: verifica si el evento contiene la cadena illegal user o la cadena invalid user.
- Description: descripción de la regla. Esta será mostrada en la interfaz.
- Group: Etiquetas que serán añadidas a la alerta.

Información sobre todas las opciones disponibles para reglas y decodificadores…
https://documentation.wazuh.com/current/user-manual/ruleset/ruleset-xml-syntax/rules.html
https://documentation.wazuh.com/current/user-manual/ruleset/ruleset-xml-syntax/decoders.html

¿Qué son los localfiles?

Los localfiles se utilizan para indicar a OSSEC la colección de datos
de logs que van a ser monitorizados por las reglas existentes.

Las fuentes de estos datos pueden ser:

- **Ficheros**: Es frecuente que las aplicaciones, ya sean del sistema o de los usuarios escriban sus logs en ficheros.
  Véase los ejemplos de la imagen.

```
<localfile>
  <log_format>syslog</log_format>
  <location>/var/ossec/logs/active-responses.log</location>
</localfile>

<localfile>
  <log_format>syslog</log_format>
  <location>/var/log/auth.log</location>
</localfile>

<localfile>
  <log_format>syslog</log_format>
  <location>/var/log/syslog</location>
</localfile>

<localfile>
  <log_format>syslog</log_format>
  <location>/var/log/dpkg.log</location>
</localfile>
```

- **Eventos del sistema**: Por ejemplo, el log de eventos de los sistemas Windows, o los broadcast del sistema en los dispositivos Android.
- **Salida de comandos**: Otro mecanismo para obtener logs es mediante el uso de comandos. Por ejemplo, podemos usar netstat para ver los puertos que están siendo usados. El sistema se encargaría de llamar al comando según la frecuencia definida.

```
<!-- Log analysis -->
<localfile>
  <log_format>command</log_format>
  <command>df -P</command>
  <frequency>360</frequency>
</localfile>

<localfile>
  <log_format>full_command</log_format>
  <command>netstat -tulpn | sed 's/\([[:alnum:]]\+\)\ \+[[:digit:]]\+\
  <alias>netstat listening ports</alias>
  <frequency>360</frequency>
</localfile>

<localfile>
  <log_format>full_command</log_format>
  <command>last -n 20</command>
  <frequency>360</frequency>
</localfile>
```

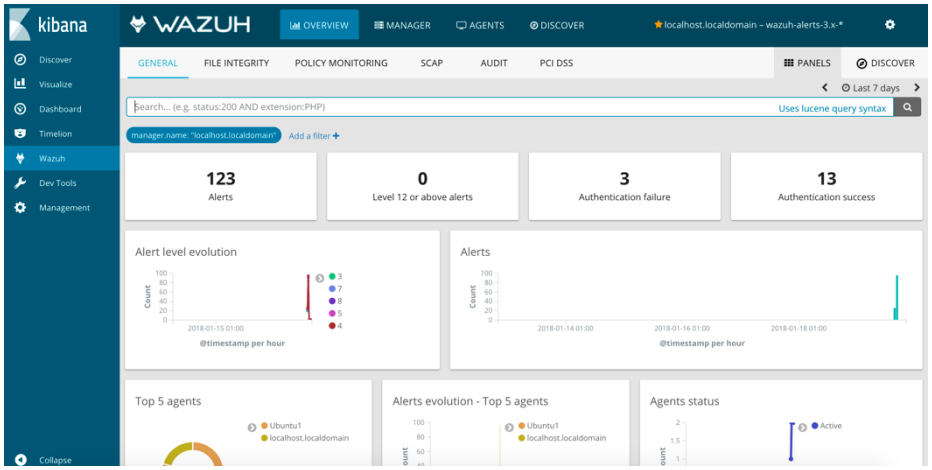
Esta configuración se encuentra en los agentes, en el fichero de configuración situado en /var/ossec/etc/ossec.conf. Puede encontrarse
documentación sobre todas las opciones disponibles en el enlace de abajo.

Enlaces interesantes…
https://documentation.wazuh.com/current/user-manual/reference/ossec-conf/localfile.html

¿Cómo se visualizan las alertas?

Como se ha observado en pasos anteriores, Wazuh ofrece una interfaz web a través de Kibana. Por lo general, la web se encuentra en el manager en el puerto 5601. Para acceder, una vez instalado correctamente, solo es necesario acceder a la dirección siguiente. http://IP_MANAGER:5601

Para empezar a trabajar será necesario clickar en la opción Wazuh, en el menú principal. Dentro encontramos opciones que permiten gestionar los diferentes agentes.



Ejemplo de uno de los dashboards de kibana que emplea Wazuh

Entre las secciones más importantes se encuentran:

- **Overview**: muestra un resumen de las diferentes alertas. Entre los recursos aparecen gráficos, contadores de ocurrencias, un buscador de eventos…
- **Manager**: permite comprobar el estado de los diferentes nodos manager disponibles en el sistema.
- **Agents**: muestra el listado de todos los agentes disponibles, así como su estado.

- **Discover**: recoge todos los eventos, permitiendo ver en detalle cada uno de ellos. Incluye un buscador y mecanismos para filtrar aquellos no relevantes.

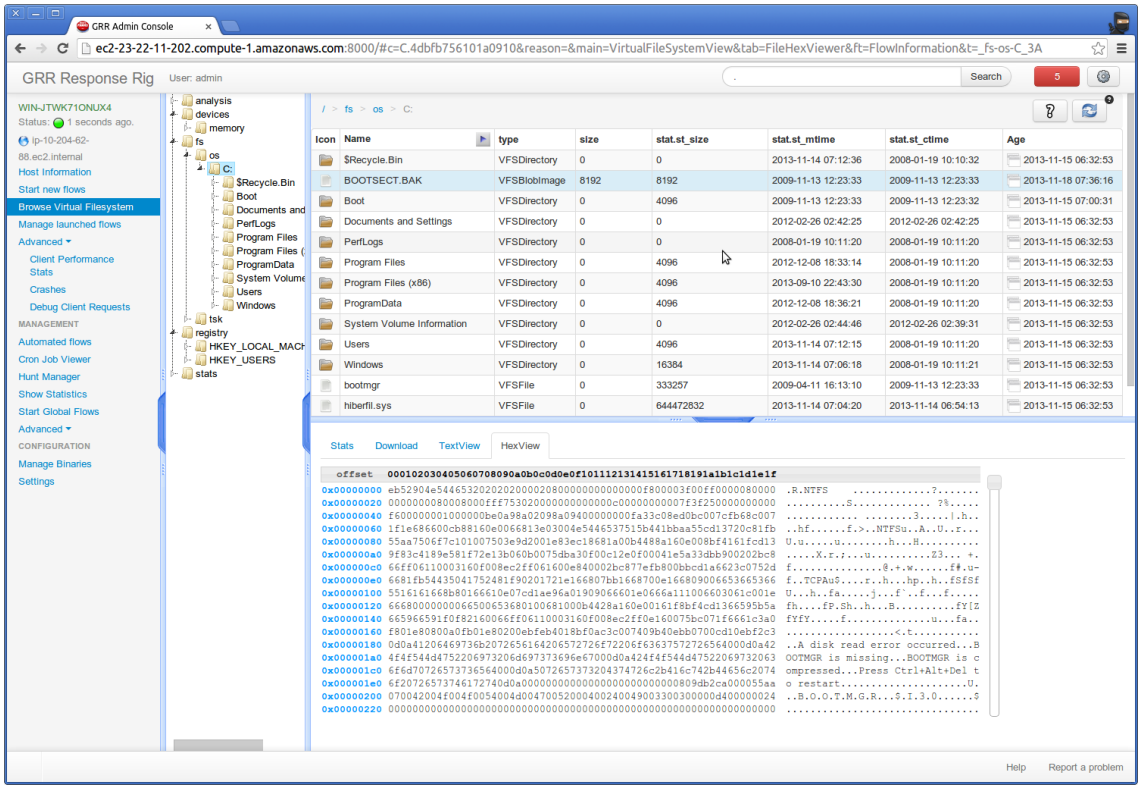
4.2.3. Otros sistemas similares a los HIDS

GRR Rapid Response

Sitio web: https://grr-doc.readthedocs.io/en/latest/
Acceso al código fuente: https://github.com/google/grr

Se trata de un proyecto OpenSource con licencia Apache 2 mantenido por la comunidad de código libre de Google. Si bien podría definirse como una plataforma de análisis forense en tiempo real, también permite análisis y detección de amenazas en tiempo real permitiendo entre otros el uso de reglas YARA para buscar amenazas por ejemplo en ficheros binarios.

GRR funciona tanto en sistemas UNIX/Linux como en Windows, funcionando en un esquema típico HIDS, es decir, se compone de agentes que permiten analizar y recoger información de las máquinas target a través de un servidor dedicado. Tanto agente como servidor están desarrollados mayormente en python



Ejemplo de visualización de la información recogida por el agente GRR en una de las máquinas analizadas

Una de sus principales ventajas es que la herramienta (especialmente el servidor) se encuentra dockerizada, permitiendo su instalación y configuración en pocos minutos.

4.2.4. Herramientas aplicadas a contenedores

A medida que durante los últimos años se ha popularizado el uso de la virtualización mediante contenedores, también lo ha hecho la creación de malware asociado. Para combatirlo, existen numerosas soluciones que permiten no solo analizar contenedores independientes, sino arquitecturas más complejas como por ejemplo aquellas basadas en Kubernetes.

Algunos ejemplos:

Kubeaudit

Se trata de una herramienta de auditoría de clusters de kubernetes, que mediante reglas de auditoría que pueden ser lanzadas permite cubrir la mayoría de las buenas prácticas de seguridad recomendadas para kubernetes.

Los comandos disponibles para la herramienta son los siguientes:

```
Available Commands:
  all        Run all audits
  allowpe    Audit containers that allow privilege escalation
  apparmor   Audit containers running without AppArmor
  autofix    Automagically fixes a manifest to be secure
  caps       Audit container for capabilities
  help       Help about any command
  image      Audit container images
  limits     Audit containers running with limits
  mountds    Audit containers that mount /var/run/docker.sock
  namespaces Audit Pods for hostNetwork, hostIPC and hostPID
  nonroot    Audit containers running as root
  np         Audit namespace network policies
  priv       Audit containers running as privileged
  rootfs     Audit containers with read only root filesystems
  sat        Audit automountServiceAccountToken = true pods against an empty (default) service account
  seccomp    Audit containers running without Seccomp
  version    Print the version number of kubeaudit
```

https://linuxsecurity.expert/tools/kubeaudit/

K8Guard

Enfocado a auditoría y hardening de clusters de kubernetes.
https://k8guard.github.io/about/violations/

Anchore

https://github.com/anchore/anchore-engine

Su estrategia es el securizado de imágenes Docker, analizando imágenes públicas con el objetivo de descubrir vulnerabilidades en su contenido.

Bibliografía:

https://es.wikipedia.org/wiki/Rootkit
https://www.incibe-cert.es/blog/malware-linux
https://wazuh.com/blog/detecting-metasploit-attacks/
https://linuxhint.com/about_rootkits_detection_and_prevention/#:~:text=Linux%20user%2Dmode%20RootKits%3A,by%20getting%20to%20a%20backdoor.
https://www.profesionalreview.com/2017/02/05/rootkits-detectarlos-linux/
https://linux-audit.com/intrusion-detection-linux-rootkits/